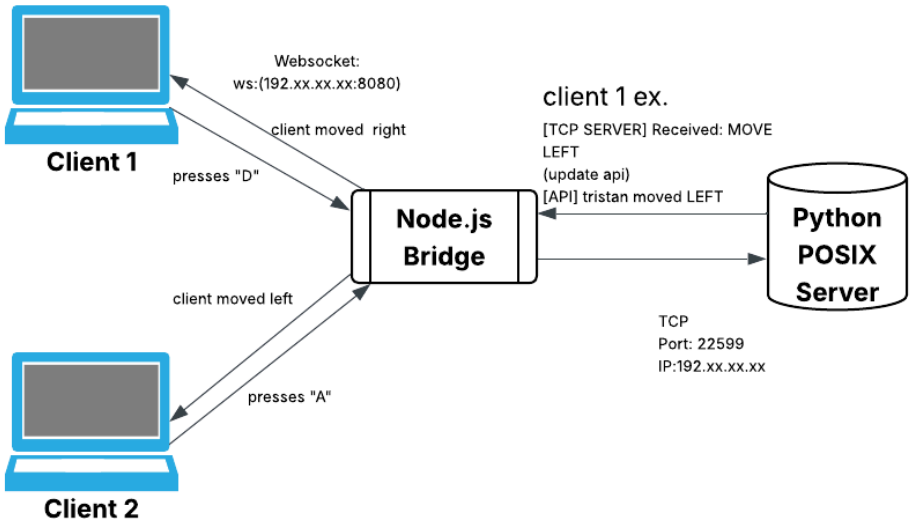
Tristan Santor
Escape the Backrooms Documentation
CSS 432 Final
12/6/2025

Overview:

The following is a writeup and summary of the development of the Escape the Backrooms Maze Game, specifically for API layer of the game which is the core aspect of the program.

Network Design(High Level): Each client sends one command per line. The server responds to the single command line string and then reacts by broadcasting to all players based on API rule.

I chose to run this game on TCP sockets which is good for the size of this game and is more reliable. Using the POSIX sockets, the game commands are sent as plain text lines since TCP will hold and deliver that information preserving the order. The following is a simple example flow the architecture used (client/server)



The socket layer receives the TCP line and strips it and forwards the string to the API layer which is mentioned below.

API Function	***How + Additions***
REGISTER <username> Announce user is available to play and store their identity on the server player list	Validate that the user name is not taken then create entries in a dictionary players[username] = socket connections[socket] = username player_state[username] = {x:0, y:0, room:None, total_score:0}
LIST_GAMES If no games exist the server returns a single line with no lobbies to display	If games is empty, return GAMES EMPTY. Otherwise iterate through all game lobbies: for game_id, info in games.items(): count = len(info["players"]) output: "<gameID> <current>/<max>" Returns formatted multi line string beginning with GAMES.

CREATE_GAME <gameID>
 	
Create a new game with max players of 5 (hardcoded) and a playerlist in that lobby
	Validate that the game ID does not already exist.
	
	Create a new lobby entry with a hardcoded maximum of 5 players:
	
	games[gameID] = {
	"players": [],
	"max": 5
	}

JOIN_GAME Validate if the game ID exists, and check if player lobby is its limit, set players starting state at x = 1, y = 1	Validate that the game exists and is not full. Recover username from the socket via: username = connections[client_socket] Add the user to the lobby: games[gameID]["players"].append(username) Set their starting game position and room: player_state[username]["room"] = gameID player_state[username]["x"] = 1 player_state[username]["y"] = 1 The server does not yet broadcast join events; the client handles its own confirmation.
EXIT_GAME Find the username from the socket, locates the game lobby the player is in and removes them	Extract the username from the socket using connections. Search every lobby to find where the player belongs.

	Remove the player from that lobby's list. Set their room to None: player_state[username]["room"] = None Broadcast to remaining players in that room: LEFT <username>
UNREGISTER Remove the user from the active player list and clean up all server states	Validate username exists in the players dictionary. If the user is inside a game lobby, remove them and broadcast: LEFT <username> Remove all internal fields: del players[username] del player_state[username] del connections[socket] Fully deletes the user.
MOVE <direction> - App Specific Protocol Defines the movement of the player and for this game the player is updated once key is pressed and updates the position in the maze.	Given UP, DOWN, LEFT, RIGHT, compute tentative new coordinates. Bounds check against the maze dimensions: if out of range: ignore Check tile type using the maze loaded from maze.json: 1 = wall = movement blocked 0 = floor = movement accepted 2 = exit = triggers win sequence
CHAT <text> - Extra Credit Allows players to chat (Press C) within the lobby and has max of 4 chats that can be shown at a time.	Ensure the message is not empty. Validate the user is in a room. Construct chat string: message = " ".join(args) Broadcast to all players in the same room: CHAT <username> <message>
SCOREBOARD - Extra Credit Valides that the client is registered, iterates over all known players in the player_state, then each player's total score is read and returns with a response of every player in their state.	Validate the user is registered. Build a multi line response that lists every known player: SCOREBOARD <user> <total_score> <user> <total_score> ... A player's score increases only when they reach the exit tile.